

K.R. Mangalam University, Gurugram

**REPORT: Data Management
Mini Toolkit (DMMT)**

Data Structures - Unit 4 Assignment

School:	School of Engineering & Technology
Program:	B.Tech (AI and ML)
Course Title:	Data Structures
Course Code:	ETCCDS202
Unit Title:	Non-Linear & Advanced Data Structures

Student Name:	Bhavya Shany
Roll Number:	2501730024
Section:	A
Semester:	2
Batch:	2025-26
Submitted To:	Mrs. Neetu Chauhan

1. Introduction

This assignment implements three important data structures used in real-world systems:

- **Binary Search Tree (BST) for ordered data storage**
- **Graph using adjacency list for network traversal**
- **Hash Table with separate chaining for fast key-value lookup**

These structures are widely used in databases, search engines, caching systems, and networking.

2. Binary Search Tree (BST)

Operations Implemented

- Insertion
- Search
- Deletion (all 3 cases)
- Inorder Traversal

Why Inorder Traversal Gives Sorted Output

In a BST:

- Left subtree contains smaller values
- Root contains the middle value
- Right subtree contains larger values

Inorder traversal follows:

Left → Root → Right

Because of this property, elements are printed in **sorted order**.

Deletion Cases Explained

1. **Leaf Node (No Child)**
The node is simply removed.
2. **Node with One Child**
The child replaces the node.

3. Node with Two Children

The node is replaced with its **inorder successor** (smallest value in right subtree).

Time Complexity (Simple Explanation)

- Average Case: **$O(\log n)$** (tree is balanced)
- Worst Case: **$O(n)$** (tree becomes like a linked list)

3. Graph (Adjacency List, BFS, DFS)

Graph Representation

The graph is stored using an **adjacency list**, where each node stores its connected neighbors and weights.

Example:

$A \rightarrow [(B,2), (C,4)]$

BFS (Breadth First Search)

- Uses a **queue**
- Visits nodes level by level

Use Case

- Finding shortest path in **unweighted graphs**
- Example: Finding minimum number of steps in a network

DFS (Depth First Search)

- Uses **recursion or stack**
- Goes deep into one branch before backtracking

Use Case

- Detecting cycles
- Solving puzzles like mazes

Difference between DFS and BFS

Sr.No.	BFS	DFS
1.	BFS is simple to implement.	DFS is complex to implement as it may suffer from infinite loop problem.
2.	BFS will perform poor is for large numbers of vertices in graph.	DFS will perform better in case of large complex graph.
3.	BFS requires more memory.	DFS requires less memory.
4.	BFS will find the shortest path if the weight on the links are uniform.	DFS can not obtain shortest path.
5.	BFS is not useful in sorting application	DFS is used in topological sorting.
6.	This algorithm works in single stage. The visited vertices are removed from the queue and then displayed at once.	This algorithm works in two stages - in the first stage the visited vertices are pushed onto the stack and later on when there is no vertex further to visit those are popped-off.
7.	If solution path is long, the whole tree must be searched up to that depth.	May settle for non-optimal solution.

Img from (<https://cse.poriyaan.in/topic/graph-traversal-methods-50550/>)

Difference Between BFS and DFS

Feature	BFS	DFS
Approach	Level-wise	Depth-wise
Data Structure	Queue	Stack/Recursion
Use	Shortest path	Cycle detection

4. Hash Table with Separate Chaining

Concept

A hash table stores data using a **hash function**:

$$(\text{index} = \text{key} \% \text{table_size})$$

What is a Collision?

A collision occurs when:

- Two different keys map to the **same index**

Example (table size = 5):

- $10 \% 5 = 0$
- $15 \% 5 = 0$

Both go to index 0 → collision

How Separate Chaining Solves It

- Each index stores a **list (chain)**
- Multiple elements can exist in same bucket

Example:

Index 0 → [(10, value), (15, value)]

Time Complexity

- Average Case: **O(1)**
- Worst Case: **O(n)** (if all elements collide)

5. Observations

- BST provides sorted data efficiently
- Graph traversal helps explore networks
- Hash tables give very fast lookup in most cases
- Collisions are unavoidable but can be handled properly

6. Conclusion

This assignment demonstrates how different data structures solve different types of problems:

- **BST → ordered storage and searching**
- **Graph → traversal and connectivity**
- **Hash Table → fast access**

Together, they form the foundation of many real-world applications.

7. References

- Data Structures textbook
- GeeksforGeeks (for concept understanding)
- Classroom notes
- Cse Poriyaan

Thank You

Efforts by-

Bhavya Shany

2501730024

